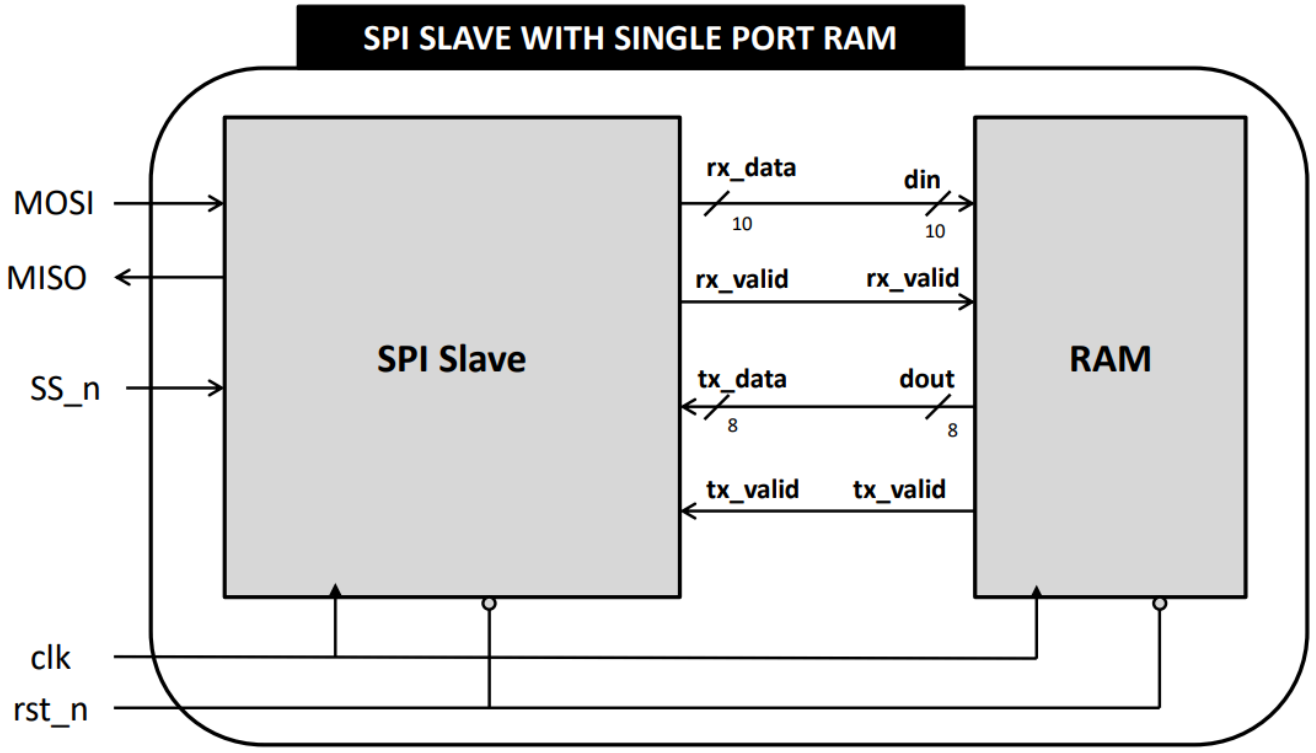
SPI Slave with Single Port RAM - RTL Design & UVM Verification

Submitted by: Rawan Mansour Abdelrahman Abdelkader



Project Overview

This project implements a complete RTL-to-verification flow for an **SPI (Serial Peripheral Interface) Slave** module integrated with a single-port RAM. The goal was to design a fully functional SPI Slave peripheral, connect it to a memory backend, and verify its correctness using a structured **UVM (Universal Verification Methodology)** testbench in SystemVerilog, simulated on QuestaSim.

The project was completed as part of a team, with each member owning a specific block or layer of the design and verification stack. The overall system consists of the SPI Slave core, the RAM interface, the UVM environment, and the simulation infrastructure.

System Architecture

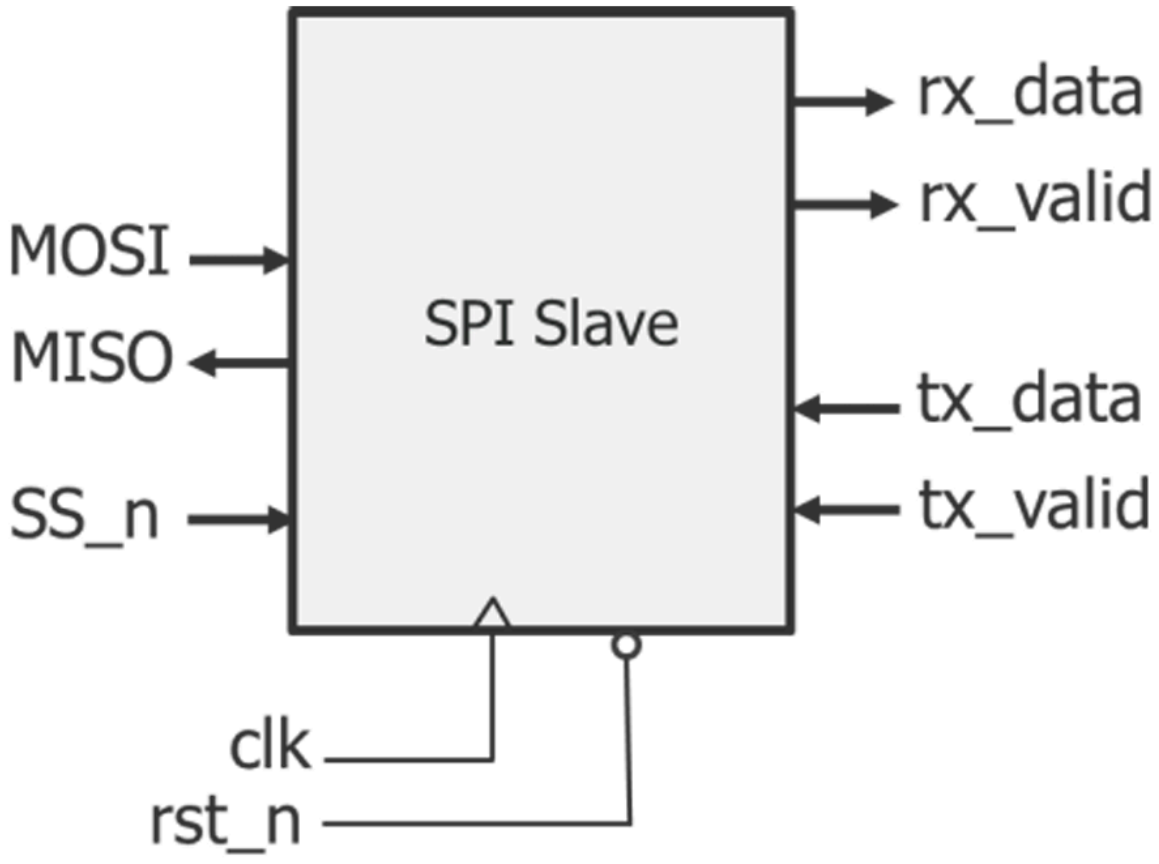
The full system is composed of two main hardware blocks connected through a set of control and data signals:

SPI Slave Core receives serial data from an SPI master over the MOSI line, decodes the incoming command, and either writes data to the RAM or reads data back from it over MISO. It is controlled entirely by the master through the chip-select signal SS_n and the serial clock.

Single-Port RAM stores and retrieves data on behalf of the slave. It receives a 10-bit input bus from the slave carrying the address and data, and returns an 8-bit output when a read is requested. The RAM is clocked synchronously alongside the slave.

Signal	Width	Direction	Description
`rx_data`	10	Slave → RAM	Address + data shifted in from master
`rx_valid`	1	Slave → RAM	Pulses high when rx_data is valid
`tx_data`	8	RAM → Slave	Data to be sent back to master
`tx_valid`	1	RAM → Slave	Asserted when tx_data is ready

SPI Slave - My Block



Overview of the SPI Slave Core Functionality

The SPI Slave core is designed to function as a peripheral device within a larger system, communicating with an SPI master controller. Its primary role is to reliably receive commands and data from the master and to transmit requested data back, all while adhering to a standard SPI protocol. The core operates based on a synchronous serial interface, where communication is framed by the Chip Select signal, `CS_n`, which is active low.

At the heart of the design is a Finite State Machine (FSM) that orchestrates all operations. This FSM governs the slave's behavior, transitioning through distinct states such as `IDLE`, `CMD`, `ADDR`, `DATA`, and `TX`. The transitions between these states are carefully controlled by the combination of the `CS_n` signal and the data arriving on the `MOSI` line from the master. A fundamental requirement is the proper handling of the active-low reset signal, `rst_n`, which must initialize all outputs and internal state machines to their known default values, ensuring a predictable starting point for operation.

The communication process begins when the master pulls the `CS_n` signal low, prompting the slave to leave its `IDLE` state. The first three bits received are treated as a command, which the slave must accurately decode to determine the requested operation, such as a write or a read. For write operations, the core is responsible for serially shifting in subsequent address and data bits, storing them correctly, and then signaling the successful reception of this data by asserting the `rx_valid` output for a single clock cycle. The integrity of the received data is paramount, and the internal `rx_data` register must be a perfect mirror of the serial bitstream that was shifted in via the `MOSI` line. This entire sequence, from the falling edge of `CS_n` to the assertion of `rx_valid`, must follow a strict and predictable timing relationship to ensure seamless integration with the controlling system.

Design Overview — SPI Slave

The SPI Slave is designed to operate as a peripheral device that communicates with an SPI master over a standard synchronous serial interface. The slave is paired with a single-port RAM, receiving write commands and responding to read requests.

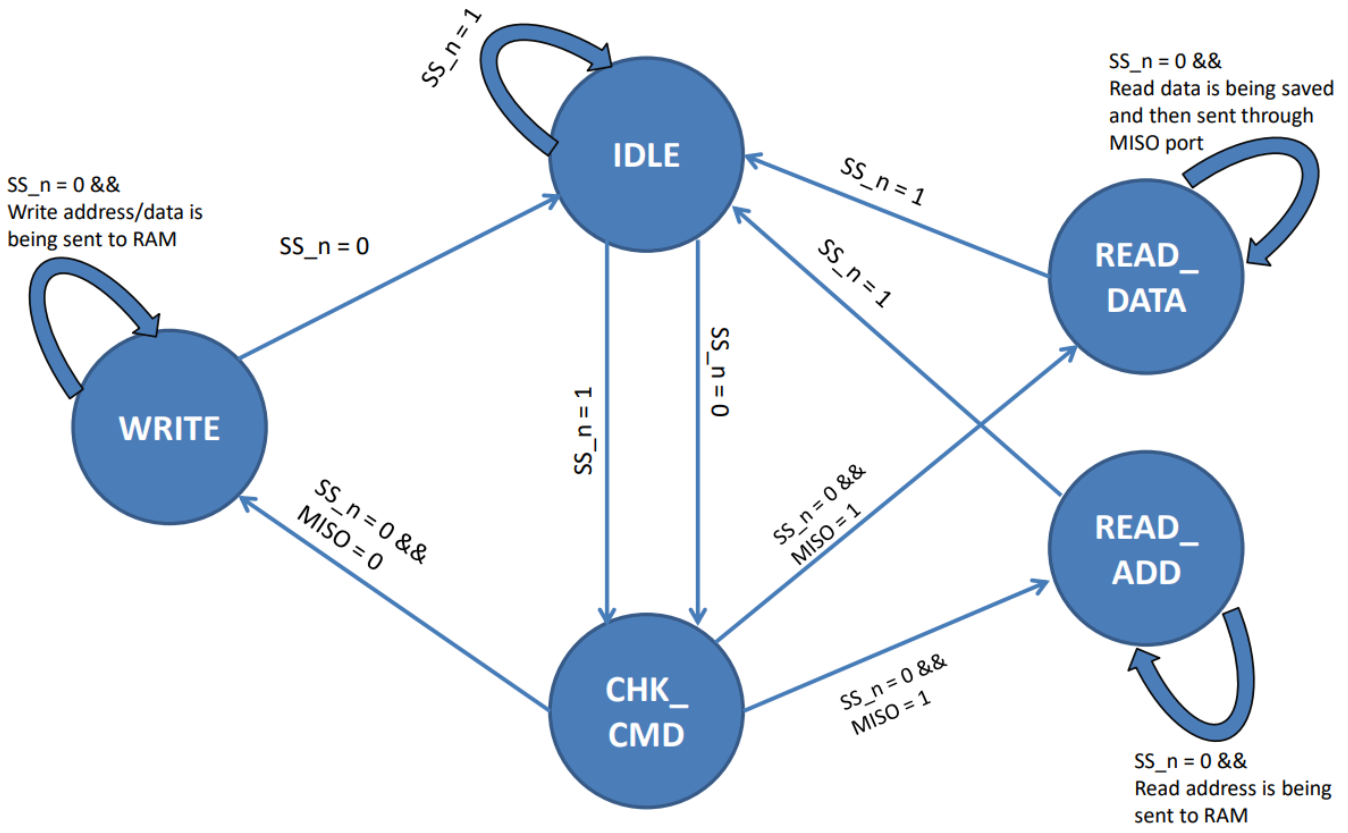
Interface Signals

Signal	Direction	Width	Description
`MOSI`	Input	1	Master Out Slave In (serial data in)
`MISO`	Output	1	Master In Slave Out (serial data out)
`SS_n`	Input	1	Chip Select, active-low
`clk`	Input	1	System clock
`rst_n`	Input	1	Synchronous active-low reset
`tx_data`	Input	8	Parallel data to shift out to master
`tx_valid`	Input	1	Asserted when `tx_data` is ready
`rx_data`	Output	10	Received serial data (address + payload)
`rx_valid`	Output	1	Single-cycle pulse on valid reception

FSM Design

I designed the FSM with 5 states to cleanly separate each phase of SPI communication:

Finite State Machine



State	Encoding	Function
`IDLE`	`3'b000`	Waits for SS_n to go low; clears rx_valid
`CHK_CMD`	`3'b010`	Decodes the first MOSI bits to determine operation
`WRITE`	`3'b001`	Shifts in 10 bits of address+data from MOSI
`READ_ADD`	`3'b011`	Shifts in address bits; sets `received_address` flag
`READ_DATA`	`3'b100`	Shifts out `tx_data` on MISO or shifts in more data

Design Bugs Fixed

Writing the SVA assertions before running the full UVM test proved essential. **Bugs 1 and 3** were caught by assertion failures before the scoreboard even had a chance to flag them — the assertions gave a cycle-accurate failure point directly in the waveform, which made root-cause analysis significantly faster than reading scoreboard error logs alone.

The dual-DUT strategy in the testbench — running the design under test alongside an independent reference model and comparing outputs cycle-by-cycle — was also a design-level decision. It meant the scoreboard needed no manually computed expected values, and any divergence between the two models was immediately flagged with full signal context.

Bug 1: Incorrect State Transition Logic

Location: CHK_CMD state in state machine
Original Buggy Code:

```
if (received_address)
    ns = READ_ADD;  // BUG: Wrong state assignment
else
    ns = READ_DATA; // BUG: Wrong state assignment
```

Fixed Code:

```
if (received_address)
    ns = READ_DATA;  // FIXED: Go to READ_DATA when address received
else
    ns = READ_ADD;   // FIXED: Go to READ_ADD when no address received
```

Impact: Caused wrong state execution and improper command handling

Bug 2: Missing Counter Reset

Location: Reset condition in always block
Original Buggy Code:

```
if (~rst_n) begin
    rx_data <= 0;
    rx_valid <= 0;
    received_address <= 0;
    MISO <= 0;
    // BUG: counter not reset - missing line
end
```

Fixed Code:

```
if (~rst_n) begin
    rx_data <= 0;
    rx_valid <= 0;
    received_address <= 0;
    MISO <= 0;
    counter <= 0;  // FIXED: Added counter reset
end
```

Impact: Counter had undefined value after reset, causing data corruption

Bug 3: Premature rx_valid Deassertion

Location: READ_DATA state, tx_valid branch
Original Buggy Code:

```
READ_DATA : begin
    if (tx_valid) begin
        rx_valid <= 0;  // BUG: rx_valid reset too early
        if (counter > 0) begin
            MISO <= tx_data[counter-1];
            counter <= counter - 1;
        end
        else begin
            received_address <= 0;
            // BUG: Missing rx_valid reset here
        end
    end
```

Fixed Code:

```
READ_DATA : begin
    if (tx_valid) begin
        if (counter > 0) begin
            MISO <= tx_data[counter-1];
            counter <= counter - 1;
        end
        else begin
            received_address <= 0;
            rx_valid <= 0;  // FIXED: rx_valid reset at proper time
        end
    end
```

Impact: Timing violation, rx_valid behavior incorrect

Bug 4: Incorrect Counter Value

Location: READ_DATA state, else branch
Original Buggy Code:

```
else begin
    if (counter > 0) begin
        rx_data[counter-1] <= MOSI;
        counter <= counter - 1;
    end
    else begin
        rx_valid <= 1;
        counter <= 8;  // BUG: Wrong counter value
    end
end
```

Fixed Code:

```
else begin
    if (counter > 0) begin
        rx_data[counter-1] <= MOSI;
        counter <= counter - 1;
    end
    else begin
        rx_valid <= 1;
        counter <= 9;  // FIXED: Correct counter value for proper operation
    end
end
```

Impact: Counter value incorrect for proper bit counting

Verification plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
SPI 0 (RESET)	Active-low reset (`rst_n`) initializes all internal registers and FSM to IDLE state.	Assert `rst_n` for a defined number of clock cycles after power-up and during runtime.	Cover that reset was entered and exited.	Check that `sdo` is high-Z, internal shift register is 0, and FSM state is IDLE.
SPI 1 (FSM Operation)	The FSM correctly transitions between IDLE, ACTIVE, and DONE states based on `cs_n` and the bit counter.	Randomize `cs_n` assertion/deassertion relative to `sck`. Vary data packet length.	Cover all FSM state transitions: IDLE->ACTIVE, ACTIVE->DONE, DONE→IDLE, ACTIVE→IDLE (early CS deassertion).	Scoreboard verifies data integrity. Assertions check that state transitions occur only on correct `sck` edges.
SPI 2 (CPOL & CPHA Modes)	The core operates correctly in all four SPI modes (CPOL=0/1, CPHA=0/1).	Randomize `cpol_i` and `cpha_i` configurations for each test. Apply `sck` and `mosi` data aligned to the selected mode.	Cover all 4 combinations of `cpol_i` and `cpha_i`. For each mode, cover successful transmission of at least one data packet.	Scoreboard samples data on the correct `sck` edge (leading/trailing) as defined by the mode. Check `sdo` is updated on the correct edge.
SPI 3 (Data Shifting IN)	Data on `mosi` is correctly shifted into the internal register, MSB-first.	Send constrained-random data on `mosi` synchronized with `sck`.	Cover various data patterns: all 0s, all 1s, alternating 01, walking 1s, and random data. Cover reception of minimum, maximum, and typical data widths.	Scoreboard compares the received parallel data (`rx_data_o`) with the expected value driven on `mosi`.
SPI 4 (Data Shifting OUT)	Data from `tx_data_i` is correctly shifted out on `sdo`, MSB-first.	Load constrained-random data into `tx_data_i` before transaction.	Cover various data patterns for transmission (same as SPL 3).	Scoreboard (or monitor) captures the serial `sdo` output and compares it to the expected serialized version of `tx_data_i`.
SPI 5 (Word Length)	The core correctly handles configurable data word lengths (`word_len_i`).	Randomize `word_len_i` between its min (e.g., 4-bit) and max (e.g., 32-bit) values for each transaction.	Cover transactions at min, max, and several intermediate word lengths.	Scoreboard dynamically adjusts to the configured word length for both reception and transmission.
SPI 6 (Control Signals)	`cs_n` starts and ends a transaction. `sck` clocks data only when `cs_n` is active.	Generate `sck` pulses while `cs_n` is high (inactive) and during active transactions.	Cover events where `sck` toggles while `cs_n` is high.	Assertions check that internal state does not change and `sdo` remains high-Z when `cs_n` is high, regardless of `sck`.

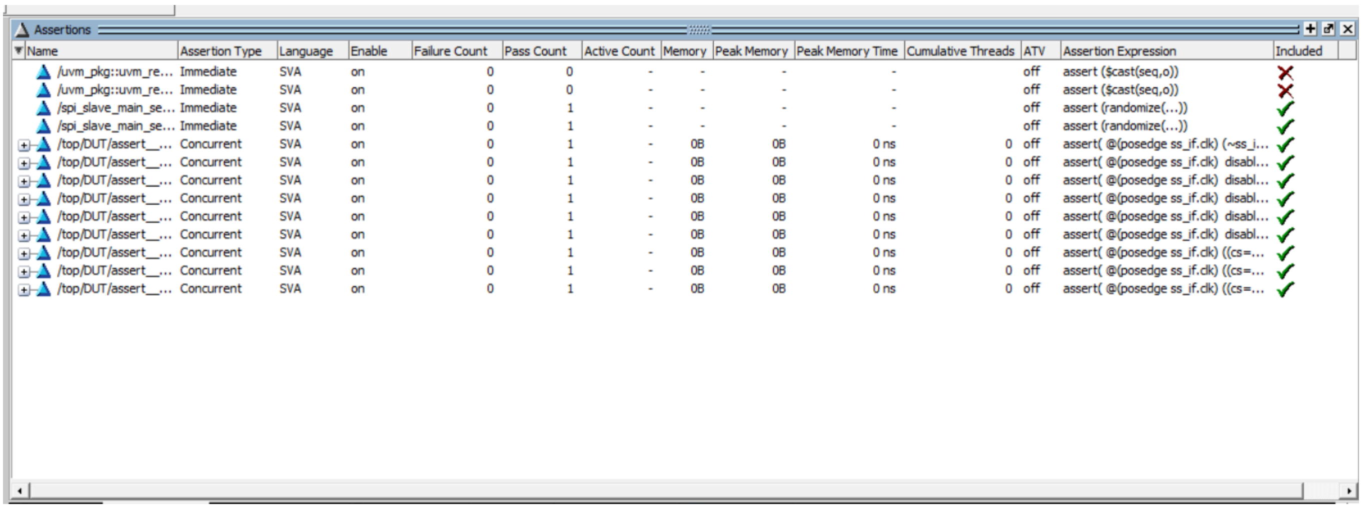
Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
SPI 7 (LSB-First Mode)	The core can be configured to shift data LSB-first via `lsb_first_i`.	Set `lsb_first_i` and send random data.	Cover successful transmission and reception of data in both MSB-first and LSB-first modes.	Scoreboard serializes/deserializes data based on the `lsb_first_i` configuration, checking bit order.
SPI 8 (Output Enable)	`sdo` is high-impedance (Z) when the slave is not selected (`cs_n` is high).	Deassert `cs_n` and monitor `sdo`.	Cover that `sdo` is Z when `cs_n=1` and has a driven value when `cs_n=0`.	Use a bus monitor or force/detect signals in the testbench to check the electrical state of `sdo`.

Assertions

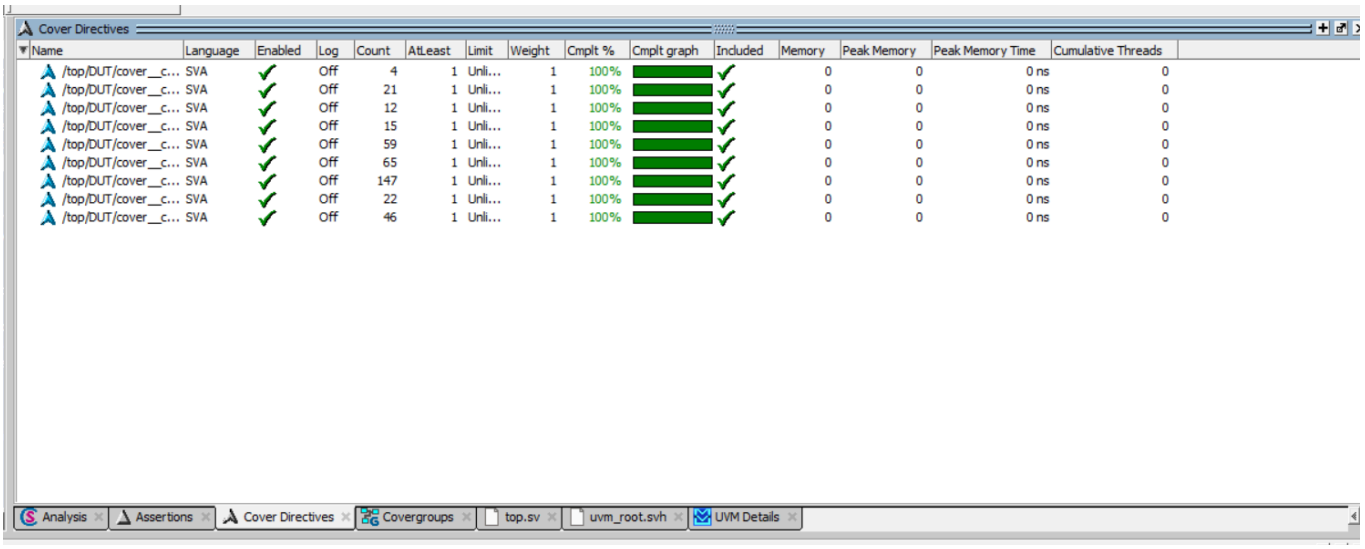
Assertion Description

Assertion Features	Assertion Property	
Reset clears all outputs	`@(posedge ss_if.clk) (~ss_if.rst_n)`	⇒ (ss_if.MISO == 0 && ss_if.rx_valid == 0 && ss_if.rx_data== 0)
Valid command asserts rx_valid	`@(posedge ss_if.clk) disable iff(~ss_if.rst_n) (write_addr_sequence or write_data_sequence or read_addr_sequence or read_data_sequence)`	⇒ ##9 ($rose(ss_if.rx_valid) && $rose(ss_if.SS_n)[→1])`
IDLE to CHK_CMD transition	`@(posedge ss_if.clk) disable iff(~ss_if.rst_n) (cs==IDLE && !ss_if.SS_n)`	⇒ (cs==CHK_CMD)
CHK_CMD to WRITE transition	`@(posedge ss_if.clk) disable iff(~ss_if.rst_n) (cs==CHK_CMD && !ss_if.SS_n && !ss_if.MOSI)`	⇒ (cs==WRITE)
CHK_CMD to READ_ADD transition	`@(posedge ss_if.clk) disable iff(~ss_if.rst_n) (cs==CHK_CMD && !ss_if.SS_n && ss_if.MOSI && !received_address)`	⇒ (cs==READ_ADD)
CHK_CMD to READ_DATA transition	`@(posedge ss_if.clk) disable iff(~ss_if.rst_n) (cs==CHK_CMD && !ss_if.SS_n && ss_if.MOSI && received_address)`	⇒ (cs==READ_DATA)
WRITE to IDLE transition	`@(posedge ss_if.clk) (cs==WRITE && (~ss_if.rst_n))`	⇒ (cs==IDLE)
READ_ADD to IDLE transition	`@(posedge ss_if.clk) (cs==READ_ADD && (~ss_if.rst_n))`	⇒ (cs==IDLE)
READ_DATA to IDLE transition	`@(posedge ss_if.clk) (cs==READ_DATA && (~ss_if.rst_n))`	⇒ (cs==IDLE)

QuestaSim Snippets for Assertion


Assertions

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
/uvm_pkg::uvm_re…	Immediate	SVA	on	0	0	-	-	-	-		off	assert ($cast(seq,o))	✗
/uvm_pkg::uvm_re…	Immediate	SVA	on	0	0	-	-	-	-		off	assert ($cast(seq,o))	✗
/spi_slave_main_se…	Immediate	SVA	on	0	1	-	-	-	-		off	assert (randomize(…))	✓
/spi_slave_main_se…	Immediate	SVA	on	0	1	-	-	-	-		off	assert (randomize(…))	✓
/top/DUT/assert__…	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge ss_if.clk) (~ss_i…	✓
/top/DUT/assert__…	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge ss_if.clk) disabl…	✓
/top/DUT/assert__…	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge ss_if.clk) disabl…	✓
/top/DUT/assert__…	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge ss_if.clk) disabl…	✓
/top/DUT/assert__…	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge ss_if.clk) disabl…	✓
/top/DUT/assert__…	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge ss_if.clk) disabl…	✓
/top/DUT/assert__…	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge ss_if.clk) ((cs=…	✓
/top/DUT/assert__…	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge ss_if.clk) ((cs=…	✓
/top/DUT/assert__…	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge ss_if.clk) ((cs=…	✓



Source Code

Do File

```
vlib work
vlog -f src_files.list +define+SIM +cover -covercells
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover
add wave -r /top/slave_interface/*
run -all
coverage save spi.ucdb -onexit
## vcover report spi.ucdb -details -annotate -output spi.txt
```

src_files.list

```
spi_slave_if.sv
spi_slave_config.sv
SPI_slave.sv
SPi_Slave_interface.v
shared_pkg.sv
spi_slave_sequence_item.sv
spi_slave_reset_sequence.sv
 spi_slave_main_sequence.sv
 spi_slave_sequencer.sv
 spi_slave_driver.sv
 spi_slave_monitor.sv
 spi_slave_agent.sv
 spi_slave_scoreboard.sv
 spi_slave_collector.sv
 spi_slave_env.sv
 spi_slave_test.sv
 top.sv
```

SPI_slave.sv

```
module SLAVE (spi_slave_if.DUT ss_if);

localparam IDLE      = 3'b000;
localparam WRITE     = 3'b001;
localparam CHK_CMD   = 3'b010;
localparam READ_ADD  = 3'b011;
localparam READ_DATA = 3'b100;

reg [3:0] counter;
reg       received_address;

reg [2:0] cs, ns;

always @(posedge ss_if.clk) begin
    if (~ss_if.rst_n) begin
        cs <= IDLE;
    end
    else begin
        cs <= ns;
    end
end

always @(*) begin
    case (cs)
        IDLE : begin
            if (ss_if.SS_n)
                ns = IDLE;
            else
                ns = CHK_CMD;
        end
        CHK_CMD : begin
            if (ss_if.SS_n)
                ns = IDLE;
            else begin
                if (~ss_if.MOSI)
                    ns = WRITE;
                else begin
                    if (received_address)
                        ns = READ_DATA;
                    else
                        ns = READ_ADD;
                end
            end
        end
        WRITE : begin
            if (ss_if.SS_n)
                ns = IDLE;
            else
                ns = WRITE;
        end
        READ_ADD : begin
            if (ss_if.SS_n)
                ns = IDLE;
            else
                ns = READ_ADD;
        end
        READ_DATA : begin
            if (ss_if.SS_n)
                ns = IDLE;
            else
                ns = READ_DATA;
        end
    endcase
end
```

```
always @(posedge ss_if.clk) begin
    if (~ss_if.rst_n) begin
        ss_if.rx_data <= 0;
        ss_if.rx_valid <= 0;
        received_address <= 0;
        ss_if.MISO <= 0;
        counter <= 0;
    end
    else begin
        case (cs)
            IDLE : begin
                ss_if.rx_valid <= 0;
            end
            CHK_CMD : begin
                counter <= 10;
            end
            WRITE : begin
                if (counter > 0) begin
                    ss_if.rx_data[counter-1] <= ss_if.MOSI;
                    counter <= counter - 1;
                end
                else begin
                    ss_if.rx_valid <= 1;
                end
            end
            READ_ADD : begin
                if (counter > 0) begin
                    ss_if.rx_data[counter-1] <= ss_if.MOSI;
                    counter <= counter - 1;
                end
                else begin
                    ss_if.rx_valid <= 1;
                    received_address <= 1;
                end
            end
            READ_DATA : begin
                if (ss_if.tx_valid) begin
                    if (counter > 0) begin
                        ss_if.MISO <= ss_if.tx_data[counter-1];
                        counter <= counter - 1;
                    end
                    else begin
                        received_address <= 0;
                        ss_if.rx_valid <= 0;
                    end
                end
                else begin
                    if (counter > 0) begin
                        ss_if.rx_data[counter-1] <= ss_if.MOSI;
                        counter <= counter - 1;
                    end
                    else begin
                        ss_if.rx_valid <= 1;
                        counter <= 9;
                    end
                end
            end
        endcase
    end
end

`ifdef SIM

// Assertions and sequences for verification
sequence write_addr_sequence;
    (ss_if.SS_n==1) ##1 (ss_if.SS_n==0) ##1 (ss_if.MOSI == 0)[*3];
```

```
endsequence

sequence write_data_sequence;
    (ss_if.SS_n==1) ##1 (ss_if.SS_n==0) ##1 (ss_if.MOSI == 0)[*2] ##1 (ss_if.MOSI==1);
endsequence

sequence read_addr_sequence;
    (ss_if.SS_n==1) ##1 (ss_if.SS_n==0) ##1 (ss_if.MOSI == 1)[*2] ##1 (ss_if.MOSI==0);
endsequence

sequence read_data_sequence;
    (ss_if.SS_n==1) ##1 (ss_if.SS_n==0) ##1 (ss_if.MOSI == 1)[*3];
endsequence

property check_rx_valid_property;
    @(posedge ss_if.clk) disable iff(~ss_if.rst_n)
    (write_addr_sequence or write_data_sequence or read_addr_sequence or read_data_sequence) |=>
    ##9 ($rose(ss_if.rx_valid) && $rose(ss_if.SS_n)[->1]);
endproperty

property check_reset_property;
    @(posedge ss_if.clk) (~ss_if.rst_n) |=> (ss_if.MISO ==0 && ss_if.rx_valid==0 && ss_if.rx_data==0);
endproperty

property check_idle_transition;
    @(posedge ss_if.clk) disable iff(~ss_if.rst_n) (cs==IDLE && !ss_if.SS_n) |=> (cs==CHK_CMD);
endproperty

property check_write_transition;
    @(posedge ss_if.clk) disable iff(~ss_if.rst_n) (cs==CHK_CMD && !ss_if.SS_n && !ss_if.MOSI) |=>
(cs==WRITE);
endproperty

property check_read_addr_transition;
    @(posedge ss_if.clk) disable iff(~ss_if.rst_n) (cs==CHK_CMD && !ss_if.SS_n && ss_if.MOSI &&
!received_address) |=> (cs==READ_ADD);
endproperty

property check_read_data_transition;
    @(posedge ss_if.clk) disable iff(~ss_if.rst_n) (cs==CHK_CMD && !ss_if.SS_n && ss_if.MOSI &&
received_address) |=> (cs==READ_DATA);
endproperty

property check_write_to_idle;
    @(posedge ss_if.clk) (cs==WRITE && (~ss_if.rst_n)) |=> (cs==IDLE);
endproperty

property check_read_addr_to_idle;
    @(posedge ss_if.clk) (cs==READ_ADD && (~ss_if.rst_n)) |=> (cs==IDLE);
endproperty

property check_read_data_to_idle;
    @(posedge ss_if.clk) (cs==READ_DATA && (~ss_if.rst_n)) |=> (cs==IDLE);
endproperty

assert property (check_reset_property);
assert property (check_rx_valid_property);
assert property (check_idle_transition);
assert property (check_write_transition);
assert property (check_read_addr_transition);
assert property (check_read_data_transition);
assert property (check_write_to_idle);
assert property (check_read_addr_to_idle);
assert property (check_read_data_to_idle);

cover property (check_reset_property);
```

```
    cover property (check_rx_valid_property);
    cover property (check_idle_transition);
    cover property (check_write_transition);
    cover property (check_read_addr_transition);
    cover property (check_read_data_transition);
    cover property (check_write_to_idle);
    cover property (check_read_addr_to_idle);
    cover property (check_read_data_to_idle);

    `endif
    endmodule
```

spi_slave_env.sv

```
    package spi_slave_env_pkg;
        import uvm_pkg::*;
        `include "uvm_macros.svh"
        import spi_slave_agent_pkg::*;
        import spi_slave_scoreboard_pkg::*;
        import spi_slave_collector_package::*;

    class spi_slave_env extends uvm_env;
        `uvm_component_utils(spi_slave_env);

        spi_slave_scoreboard scoreboard_component;
        spi_slave_coverage coverage_component;
        spi_slave_agent agent_component;

        function new(string name = "spi_slave_env", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            agent_component = spi_slave_agent::type_id::create("agent_component", this);
            scoreboard_component = spi_slave_scoreboard::type_id::create("scoreboard_component", this);
            coverage_component = spi_slave_coverage::type_id::create("coverage_component", this);
        endfunction

        function void connect_phase(uvm_phase phase);
            agent_component.agent_analysis_port.connect(scoreboard_component.scoreboard_export);
            agent_component.agent_analysis_port.connect(coverage_component.coverage_export);
        endfunction
    endclass
    endpackage
```

spi_slave_test.sv

```
package spi_slave_test_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import spi_slave_env_pkg::*;
    import spi_slave_reset_sequence_package::*;
    import spi_slave_main_sequence_package::*;
    import spi_slave_config_pkg::*;

class spi_slave_test extends uvm_test;
    `uvm_component_utils(spi_slave_test);

    spi_slave_env test_environment;
    spi_slave_config test_configuration;
    spi_slave_reset_sequence reset_sequence;
    spi_slave_main_sequence main_test_sequence;

    function new(string name = "spi_slave_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        test_environment = spi_slave_env::type_id::create("test_environment", this);
        test_configuration = spi_slave_config::type_id::create("test_configuration", this);
        reset_sequence = spi_slave_reset_sequence::type_id::create("reset_sequence", this);
        main_test_sequence = spi_slave_main_sequence::type_id::create("main_test_sequence", this);

        if (!uvm_config_db #(virtual spi_slave_if)::get(this, "", "SPI_IF",
test_configuration.config_interface))
            `uvm_fatal("TEST_CONFIG", "Virtual interface configuration failed")

        uvm_config_db #(spi_slave_config)::set(this, "*", "CFG", test_configuration);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        phase.raise_objection(this);

        `uvm_info("TEST_EXECUTION", "Starting reset sequence", UVM_LOW)
        reset_sequence.start(test_environment.agent_component.sequencer);
        `uvm_info("TEST_EXECUTION", "Reset sequence completed", UVM_LOW)

        `uvm_info("TEST_EXECUTION", "Beginning main stimulus generation", UVM_LOW)
        main_test_sequence.start(test_environment.agent_component.sequencer);
        `uvm_info("TEST_EXECUTION", "Stimulus generation finished", UVM_LOW)

        phase.drop_objection(this);
    endtask
endclass: spi_slave_test
endpackage
```

spi_slave_main_sequence.sv

```
package spi_slave_main_sequence_package;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import spi_slave_seq_item_package::*;
    import shared_pkg::*;

class spi_slave_main_sequence extends uvm_sequence #(spi_slave_seq_item);
    `uvm_object_utils(spi_slave_main_sequence);

    spi_slave_seq_item sequence_item;

    function new(string name = "spi_slave_main_sequence");
        super.new(name);
    endfunction

    task body();
        sequence_item = spi_slave_seq_item::type_id::create("sequence_item");

        // First sequence block: general operations
        repeat(1000) begin
            start_item(sequence_item);

            // Configure constraints for general cases
            sequence_item.reset_constraint.constraint_mode(1);
            sequence_item.ss_n_general_cases.constraint_mode(1);
            sequence_item.mosi_data_constraint.constraint_mode(1);
            sequence_item.ss_n_read_data_case.constraint_mode(0);
            sequence_item.tx_valid_control.constraint_mode(0);

            if (transaction_counter == 0) begin
                sequence_item.random_array.rand_mode(1);
                $display("Random Array = %b, Time = %t, Counter = %d",
                        sequence_item.random_array, $time, transaction_counter);
            end
            else begin
                sequence_item.random_array.rand_mode(0);
            end

            assert(sequence_item.randomize() with {
                sequence_item.random_array[0:2] inside {3'b000, 3'b001, 3'b110};
            });

            sequence_item.update_transaction_counter();
            finish_item(sequence_item);
        end

        // Second sequence block: read data operations
        repeat(1000) begin
            start_item(sequence_item);

            // Configure constraints for read data cases
            sequence_item.reset_constraint.constraint_mode(1);
            sequence_item.ss_n_general_cases.constraint_mode(0);
            sequence_item.mosi_data_constraint.constraint_mode(0);
            sequence_item.ss_n_read_data_case.constraint_mode(1);
            sequence_item.tx_valid_control.constraint_mode(1);

            if (read_operation_count == 0) begin
                sequence_item.random_array.rand_mode(1);
                $display("Random Array = %b, Time = %t, Counter = %d",
                        sequence_item.random_array, $time, read_operation_count);
            end
            else begin
                sequence_item.random_array.rand_mode(0);
            end
```

```
            assert(sequence_item.randomize() with {
                sequence_item.random_array[0:2] inside {3'b111};
            });

            sequence_item.update_read_operation_counter();
            finish_item(sequence_item);
        end
    endtask

endclass
endpackage
```

spi_slave_if.sv

```
interface spi_slave_if (input bit clk);
    // Input signals to DUT
    logic           MOSI, rst_n, SS_n, tx_valid;
    logic     [7:0] tx_data;

    // Output signals from DUT
    logic   [9:0] rx_data;
    logic         rx_valid, MISO;

    // Reference model outputs for comparison
    logic   [9:0] rx_data_ref;
    logic         rx_valid_ref, MISO_ref;

    modport DUT(
        input clk, MOSI, rst_n, SS_n, tx_data, tx_valid,
        output MISO, rx_data, rx_valid
    );

endinterface : spi_slave_if
```

spi_slave_sequencer.sv

```
package spi_slave_sequencer_package;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import spi_slave_seq_item_package::*;

class spi_slave_sequencer extends uvm_sequencer #(spi_slave_seq_item);
    `uvm_component_utils(spi_slave_sequencer)

    function new(string name = "spi_slave_sequencer", uvm_component parent = null);
        super.new(name, parent);
    endfunction

endclass
endpackage
```

spi_slave_driver.sv

```
package spi_slave_driver_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import spi_slave_seq_item_package::*;
    import shared_pkg::*;

class spi_slave_driver extends uvm_driver #(spi_slave_seq_item);
    `uvm_component_utils(spi_slave_driver);

    virtual spi_slave_if driver_interface;
    spi_slave_seq_item current_item;

    function new(string name = "spi_slave_driver", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            current_item = spi_slave_seq_item::type_id::create("current_item");
            seq_item_port.get_next_item(current_item);

            // Drive signals to interface
            driver_interface.rst_n = current_item.rst_n;
            driver_interface.SS_n = current_item.SS_n;
            driver_interface.MOSI = current_item.MOSI;
            driver_interface.tx_valid = current_item.tx_valid;
            driver_interface.tx_data = current_item.tx_data;

            @(negedge driver_interface.clk);
            seq_item_port.item_done();

            `uvm_info("DRIVER_RUN", current_item.convert2string_stimulus(), UVM_HIGH)
        end
    endtask

endclass
endpackage
```

spi_slave_sequence_item.sv

```
package spi_slave_seq_item_package;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import shared_pkg::*;

class spi_slave_seq_item extends uvm_sequence_item;
    `uvm_object_utils(spi_slave_seq_item);

    // Signal declarations
    rand logic [7:0] tx_data;
    rand logic SS_n;
    rand logic rst_n;
    rand logic tx_valid;
    rand bit [0:10] random_array;

    logic [9:0] rx_data, rx_data_ref;
    logic rx_valid, MISO, MISO_ref, rx_valid_ref;
    logic MOSI;

    function new(string name = "spi_slave_seq_item");
        super.new(name);
    endfunction

    // Reset constraint
    constraint reset_constraint {
        rst_n dist {1 :/ 98, 0 :/ 2};
    }

    // SS_n constraint for all cases except read data
    constraint ss_n_general_cases {
        if (random_array[0:2] inside {3'b000, 3'b001, 3'b110} && transaction_counter % 14 != 0) {
            SS_n == 0;
        }
        else {
            SS_n == 1;
        }
    }

    // SS_n constraint specifically for read data
    constraint ss_n_read_data_case {
        if (random_array[0:2] inside {3'b111} && read_operation_count % 24 != 0) {
            SS_n == 0;
        }
        else {
            SS_n == 1;
        }
    }

    // tx_valid constraint for read data operations
    constraint tx_valid_control {
        if (random_array[0:2] == 3'b111 && read_operation_count == 23) {
            tx_valid == 1;
        }
        if (random_array[0:2] != 3'b111) {
            tx_valid == 0;
        }
    }

    // Random array constraint for MOSI generation
    constraint mosi_data_constraint {
        if (previous_SS_state && !SS_n) {
            random_array[0:2] inside {3'b000, 3'b001, 3'b110, 3'b111};
        }
    }
```

```
    function void post_randomize();
        previous_SS_state = SS_n;
        if (transaction_counter < 11)
            MOSI = random_array[transaction_counter];
    endfunction

    function void update_transaction_counter();
        transaction_counter++;
        if (transaction_counter == 14)
            transaction_counter = 0;
    endfunction

    function void update_read_operation_counter();
        read_operation_count++;
        if (read_operation_count == 24)
            read_operation_count = 0;
    endfunction

    function string convert2string();
        return $sformatf("%s reset = 0b%0b, mosi = %b, miso = %b, ss_n = %b",
                         super.convert2string(), rst_n, MOSI, MISO, SS_n);
    endfunction

    function string convert2string_stimulus();
        return $sformatf("reset = 0b%0b, mosi = %b, ss_n = %b",
                         rst_n, MOSI, SS_n);
    endfunction

endclass
endpackage
```

SPi_Slave_interface.v

```
module SPI_Slave(
    input MOSI, SS_n, clk, rst_n, tx_valid,
    input [7:0] tx_data,
    output reg MISO, rx_valid,
    output reg [9:0] rx_data
);
    parameter ST_IDLE = 3'b000;
    parameter ST_CHECK_CMD = 3'b001;
    parameter ST_WRITE = 3'b010;
    parameter ST_READ_ADDR = 3'b011;
    parameter ST_READ_DATA = 3'b100;

    reg address_read_flag;
    reg [3:0] bit_counter;
    reg [2:0] current_state, next_state;

    always @(posedge clk) begin
        if (~rst_n)
            current_state <= ST_IDLE;
        else
            current_state <= next_state;
    end

    always @(*) begin
        case(current_state)
            ST_IDLE: begin
                if (SS_n)
                    next_state = ST_IDLE;
                else
                    next_state = ST_CHECK_CMD;
            end
            ST_CHECK_CMD: begin
                if (SS_n)
                    next_state = ST_IDLE;
                else begin
                    if (~MOSI) begin
                        next_state = ST_WRITE;
                    end
                    else begin
                        casex(address_read_flag)
                            1'b0: next_state = ST_READ_ADDR;
                            1'b1: next_state = ST_READ_DATA;
                            1'bx: next_state = ST_READ_ADDR;
                        endcase
                    end
                end
            end
            ST_WRITE: begin
                if (SS_n == 0)
                    next_state = ST_WRITE;
                else begin
                    next_state = ST_IDLE;
                end
            end
            ST_READ_ADDR: begin
                if (SS_n == 0) begin
                    next_state = ST_READ_ADDR;
                end
                else begin
                    next_state = ST_IDLE;
                end
            end
            ST_READ_DATA: begin
                if (SS_n == 0)
                    next_state = ST_READ_DATA;
```

```
                else begin
                    next_state = ST_IDLE;
                end
            end
        endcase
    end

    always @(posedge clk) begin
        if (~rst_n) begin
            rx_data <= 0;
            rx_valid <= 0;
            address_read_flag <= 0;
            MISO <= 0;
        end
        else begin
            case(current_state)
                ST_IDLE: rx_valid <= 0;
                ST_CHECK_CMD: begin
                    bit_counter <= 10;
                end
                ST_WRITE: begin
                    if (bit_counter > 0) begin
                        rx_data[bit_counter-1] <= MOSI;
                        bit_counter <= bit_counter - 1;
                    end
                    else begin
                        rx_valid <= 1;
                    end
                end
                ST_READ_ADDR: begin
                    if (bit_counter > 0) begin
                        rx_data[bit_counter-1] <= MOSI;
                        bit_counter <= bit_counter - 1;
                    end
                    else begin
                        rx_valid <= 1;
                        address_read_flag <= 1;
                    end
                end
                ST_READ_DATA: begin
                    if (tx_valid) begin
                        if (bit_counter > 0) begin
                            MISO <= tx_data[bit_counter-1];
                            bit_counter <= bit_counter - 1;
                        end
                        else begin
                            address_read_flag <= 0;
                            rx_valid <= 0;
                        end
                    end
                    else begin
                        if (bit_counter > 0) begin
                            rx_data[bit_counter-1] <= MOSI;
                            bit_counter <= bit_counter - 1;
                        end
                        else begin
                            rx_valid <= 1;
                            bit_counter <= 9;
                        end
                    end
                end
            endcase
        end
    end
endmodule
```

spi_slave_agent.sv

```
package spi_slave_agent_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import spi_slave_seq_item_package::*;
    import spi_slave_config_pkg::*;
    import spi_slave_driver_pkg::*;
    import spi_slave_monitor_pkg::*;
    import spi_slave_sequencer_package::*;

class spi_slave_agent extends uvm_agent;
    `uvm_component_utils(spi_slave_agent)

    spi_slave_config agent_config;
    spi_slave_driver driver;
    spi_slave_sequencer sequencer;
    spi_slave_monitor monitor;
    uvm_analysis_port #(spi_slave_seq_item) agent_analysis_port;

    function new(string name = "spi_slave_agent", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(spi_slave_config)::get(this, "", "CFG", agent_config)) begin
            `uvm_fatal("AGENT_BUILD", "Configuration object retrieval failed")
        end
        sequencer = spi_slave_sequencer::type_id::create("sequencer", this);
        driver = spi_slave_driver::type_id::create("driver", this);
        monitor = spi_slave_monitor::type_id::create("monitor", this);
        agent_analysis_port = new("agent_analysis_port", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        driver.driver_interface = agent_config.config_interface;
        monitor.monitor_interface = agent_config.config_interface;
        driver.seq_item_port.connect(sequencer.seq_item_export);
        monitor.monitor_analysis_port.connect(agent_analysis_port);
    endfunction
endclass
endpackage
```

top.sv

```
import uvm_pkg::*;
`include "uvm_macros.svh"
import spi_slave_test_pkg::*;
import spi_slave_seq_item_package::*;
import shared_pkg::*;

module top();
    bit clk;
    initial begin
        forever #2 clk = ~clk;
    end

    spi_slave_if slave_interface(clk);

    // Design Under Test with interface connection
    SLAVE DUT(slave_interface);

    // Reference model with individual signal connections for comparison
    SPI_Slave reference_dut(
        .MOSI(slave_interface.MOSI),
        .SS_n(slave_interface.SS_n),
        .clk(slave_interface.clk),
        .rst_n(slave_interface.rst_n),
        .rx_data(slave_interface.rx_data_ref),
        .tx_valid(slave_interface.tx_valid),
        .tx_data(slave_interface.tx_data),
        .MISO(slave_interface.MISO_ref),
        .rx_valid(slave_interface.rx_valid_ref)
    );

    initial begin
        uvm_config_db #(virtual spi_slave_if)::set(null, "uvm_test_top", "SPI_IF", slave_interface);
        run_test("spi_slave_test");
    end
endmodule
```

spi_slave_monitor.sv

```
package spi_slave_monitor_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import spi_slave_seq_item_package::*;
    import shared_pkg::*;

class spi_slave_monitor extends uvm_monitor;
    `uvm_component_utils(spi_slave_monitor);

    virtual spi_slave_if monitor_interface;
    spi_slave_seq_item monitored_item;
    uvm_analysis_port #(spi_slave_seq_item) monitor_analysis_port;

    function new(string name = "spi_slave_monitor", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        monitor_analysis_port = new("monitor_analysis_port", this);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            monitored_item = spi_slave_seq_item::type_id::create("monitored_item");
            @(negedge monitor_interface.clk);

            // Capture all interface signals
            monitored_item.rst_n = monitor_interface.rst_n;
            monitored_item.MOSI = monitor_interface.MOSI;
            monitored_item.SS_n = monitor_interface.SS_n;
            monitored_item.MISO = monitor_interface.MISO;
            monitored_item.MISO_ref = monitor_interface.MISO_ref;
            monitored_item.tx_valid = monitor_interface.tx_valid;
            monitored_item.tx_data = monitor_interface.tx_data;
            monitored_item.rx_valid = monitor_interface.rx_valid;
            monitored_item.rx_data = monitor_interface.rx_data;
            monitored_item.rx_valid_ref = monitor_interface.rx_valid_ref;
            monitored_item.rx_data_ref = monitor_interface.rx_data_ref;

            monitor_analysis_port.write(monitored_item);
            `uvm_info("MONITOR_RUN", monitored_item.convert2string_stimulus(), UVM_HIGH)
        end
    endtask

endclass
endpackage
```

spi_slave_scoreboard.sv

```
package spi_slave_scoreboard_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import spi_slave_seq_item_package::*;

class spi_slave_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(spi_slave_scoreboard);

    uvm_analysis_export #(spi_slave_seq_item) scoreboard_export;
    uvm_tlm_analysis_fifo #(spi_slave_seq_item) scoreboard_fifo;
    spi_slave_seq_item scoreboard_item;
    int error_counter = 0;
    int success_counter = 0;

    function new(string name = "spi_slave_scoreboard", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        scoreboard_export = new("scoreboard_export", this);
        scoreboard_fifo = new("scoreboard_fifo", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        scoreboard_export.connect(scoreboard_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            scoreboard_fifo.get(scoreboard_item);
            if (scoreboard_item.MISO != scoreboard_item.MISO_ref ||
                scoreboard_item.rx_valid != scoreboard_item.rx_valid_ref ||
                scoreboard_item.rx_data != scoreboard_item.rx_data_ref) begin

                `uvm_error("SCOREBOARD_CHECK", $sformatf("Mismatch detected! DUT Output: %s Reference Output:
MISO=0b%0b",
                    scoreboard_item.convert2string(), scoreboard_item.MISO_ref))
                error_counter++;
            end
            else begin
                `uvm_info("SCOREBOARD_CHECK", $sformatf("Transaction verified successfully: %s",
                    scoreboard_item.convert2string()), UVM_HIGH)
                success_counter++;
            end
        end
    endtask

    function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        `uvm_info("FINAL_REPORT", $sformatf("Successful transactions: %0d", success_counter), UVM_MEDIUM)
        `uvm_info("FINAL_REPORT", $sformatf("Failed transactions: %0d", error_counter), UVM_MEDIUM)
    endfunction

endclass
endpackage
```

spi_slave_collector.sv

```
package spi_slave_collector_package;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import spi_slave_seq_item_package::*;
    import shared_pkg::*;

class spi_slave_coverage extends uvm_component;
    `uvm_component_utils(spi_slave_coverage)

    uvm_analysis_export #(spi_slave_seq_item) coverage_export;
    uvm_tlm_analysis_fifo #(spi_slave_seq_item) coverage_fifo;
    spi_slave_seq_item coverage_item;

    covergroup functional_coverage;
        receiver_data: coverpoint coverage_item.rx_data[9:8];
        ss_n_general_cases : coverpoint coverage_item.SS_n {
            bins transaction_complete = (1 => 0[*13] => 1);
        }
        ss_n_read_operations : coverpoint coverage_item.SS_n {
            bins read_transaction = (1 => 0[*23] => 1);
        }
        ss_n_active : coverpoint coverage_item.SS_n {
            bins slave_select_active = {0};
        }
        mosi_patterns: coverpoint coverage_item.MOSI {
            bins write_address_cmd = (0 => 0 => 0);
            bins write_data_cmd = (0 => 0 => 1);
            bins read_address_cmd = (1 => 1 => 0);
            bins read_data_cmd = (1 => 1 => 1);
        }
        cross_mosi_ss_combinations : cross mosi_patterns, ss_n_active {
            option.cross_auto_bin_max = 0;
            bins write_addr_active = binsof(ss_n_active.slave_select_active) &&
binsof(mosi_patterns.write_address_cmd);
            bins write_data_active = binsof(ss_n_active.slave_select_active) &&
binsof(mosi_patterns.write_data_cmd);
            bins read_addr_active = binsof(ss_n_active.slave_select_active) &&
binsof(mosi_patterns.read_address_cmd);
            bins read_data_active = binsof(ss_n_active.slave_select_active) &&
binsof(mosi_patterns.read_data_cmd);
        }
    endgroup

    function new(string name = "spi_slave_coverage", uvm_component parent = null);
        super.new(name, parent);
        functional_coverage = new();
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        coverage_export = new("coverage_export", this);
        coverage_fifo = new("coverage_fifo", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        coverage_export.connect(coverage_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            coverage_fifo.get(coverage_item);
            functional_coverage.sample();
        end
```

```
    endtask

endclass
endpackage
```

spi_slave_config.sv

```
package spi_slave_config_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

class spi_slave_config extends uvm_object;
    `uvm_object_utils(spi_slave_config)

    virtual spi_slave_if config_interface;

    function new(string name = "spi_slave_config");
        super.new(name);
    endfunction

endclass
endpackage
```

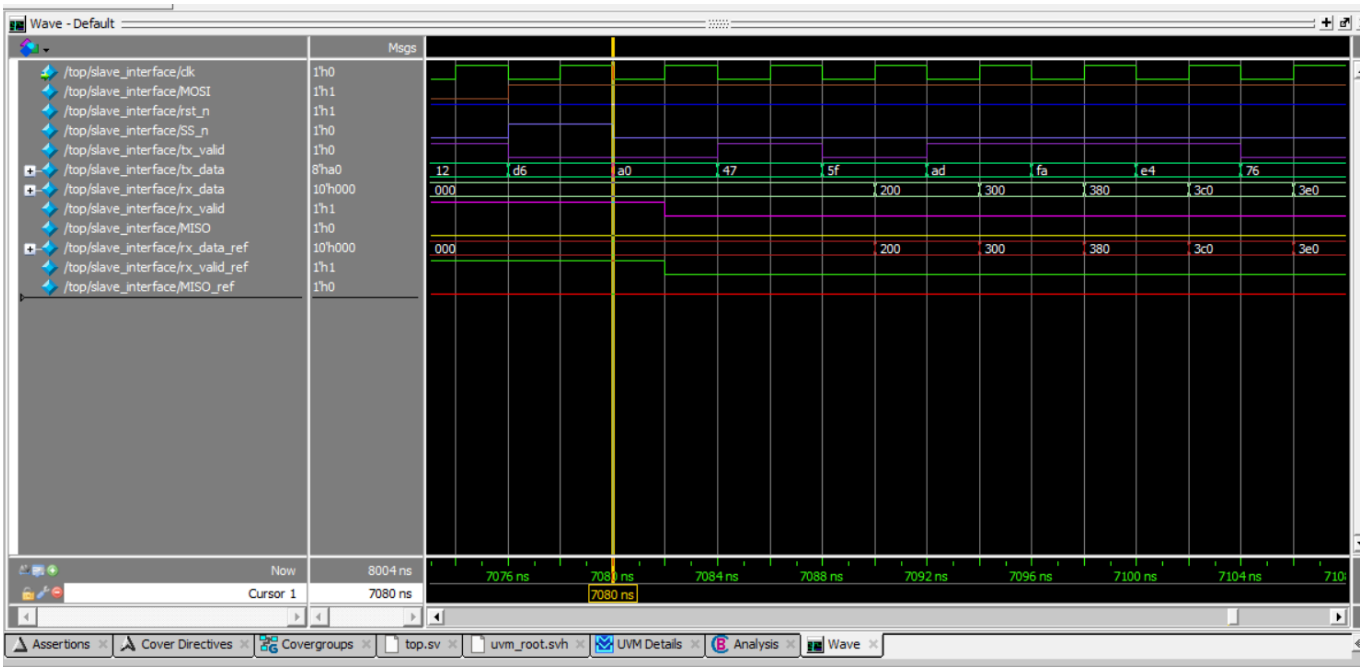
spi_slave_reset_sequence.sv

```
package spi_slave_reset_sequence_package;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import spi_slave_seq_item_package::*;

class spi_slave_reset_sequence extends uvm_sequence #(spi_slave_seq_item);
    `uvm_object_utils(spi_slave_reset_sequence);

    spi_slave_seq_item reset_item;

    function new(string name = "spi_slave_reset_sequence");
        super.new(name);
    endfunction

    task body();
        reset_item = spi_slave_seq_item::type_id::create("reset_item");
        start_item(reset_item);

        // Apply reset conditions
        reset_item.rst_n = 1;
        reset_item.rx_valid = 0;
        reset_item.tx_valid = 0;
        reset_item.SS_n = 1;

        finish_item(reset_item);
    endtask

endclass
endpackage
```

shared_pkg.sv

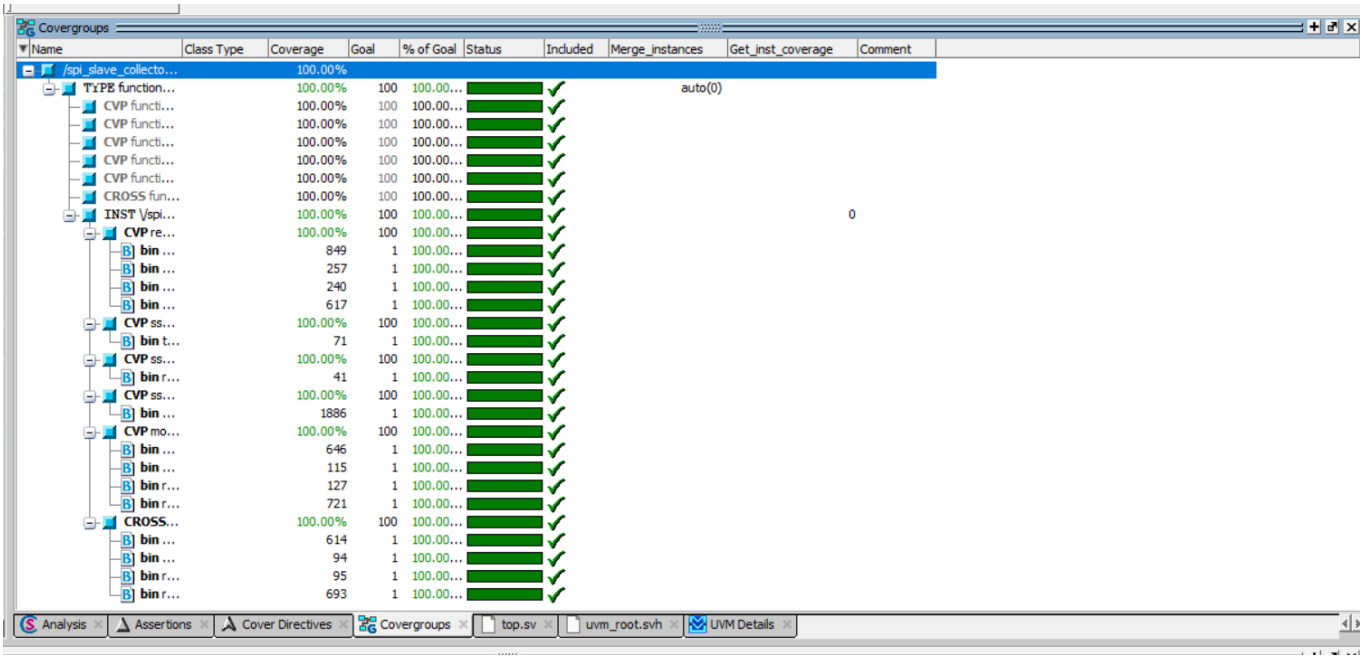
```
package shared_pkg;
    int transaction_counter = 0;
    int read_operation_count = 0;
    logic previous_SS_state = 1;
endpackage
```

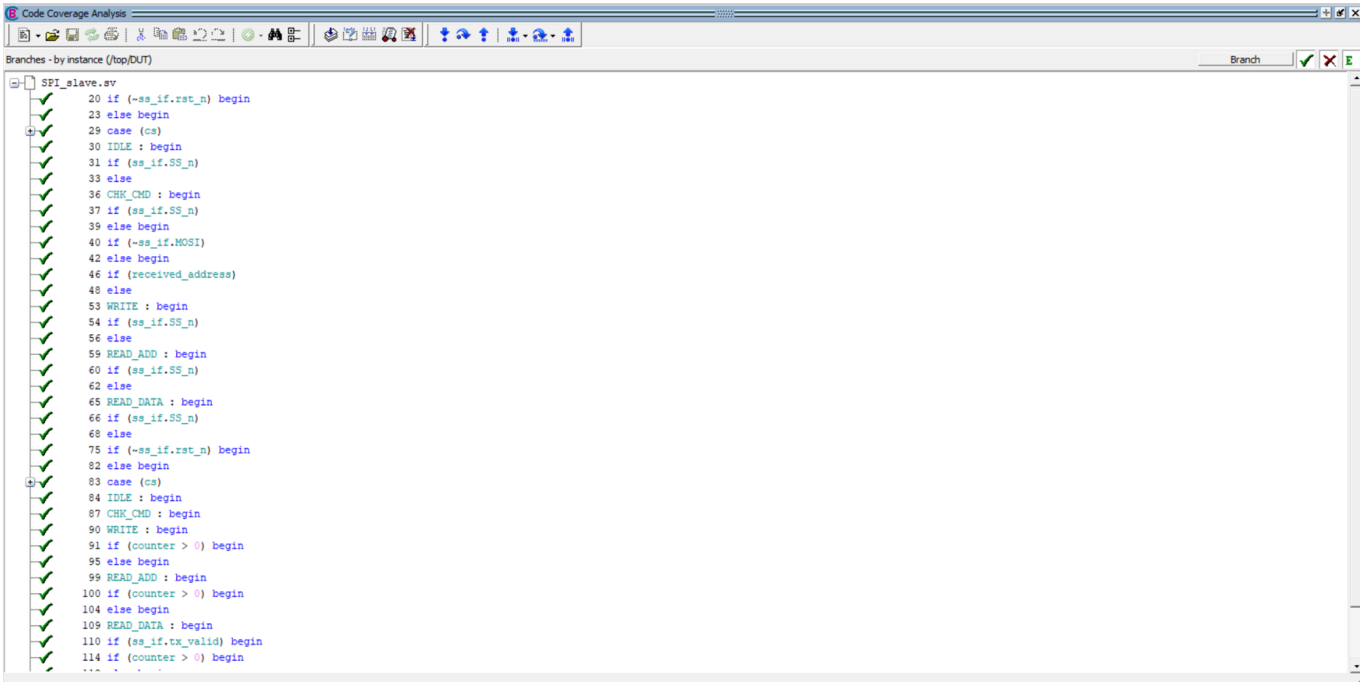
Wave Form

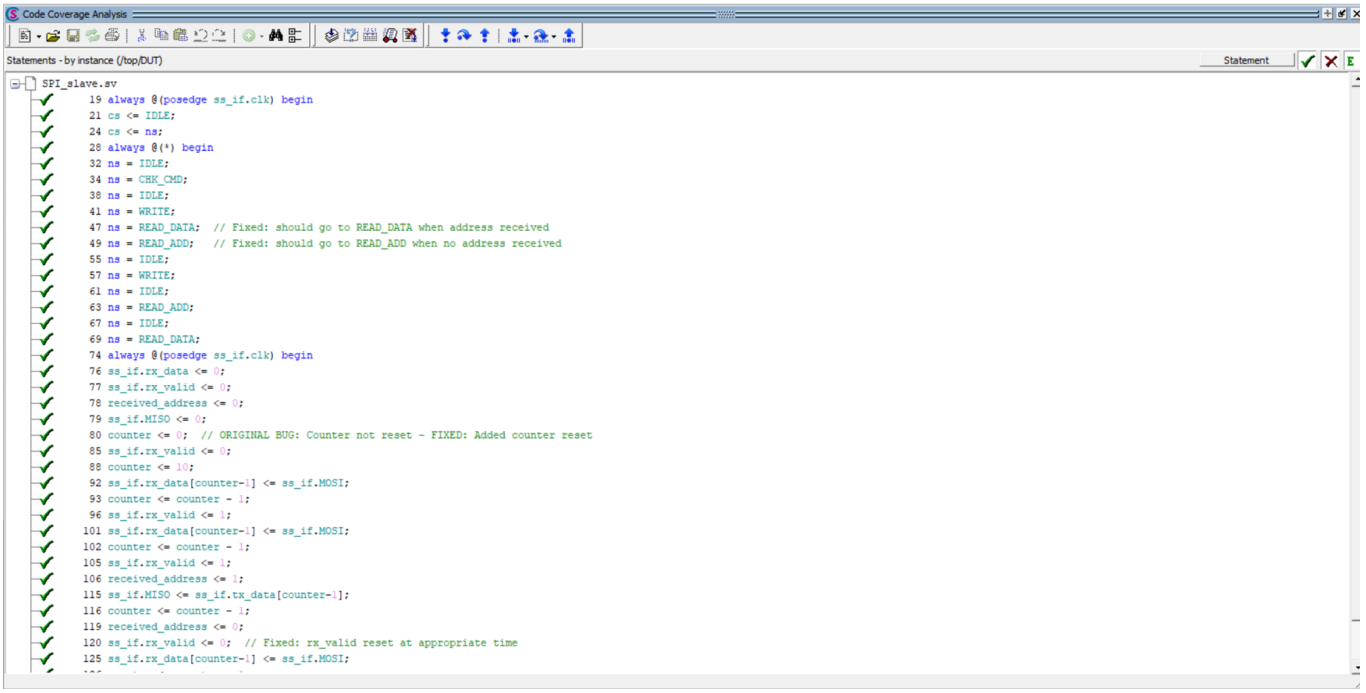


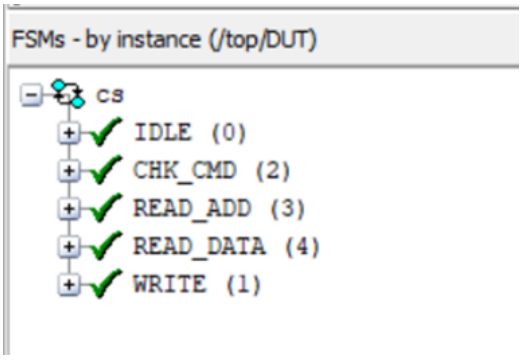
Code Coverage

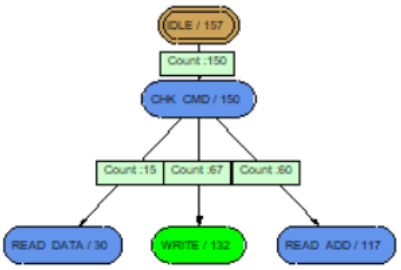
QuestaSim Snippets for Coverage

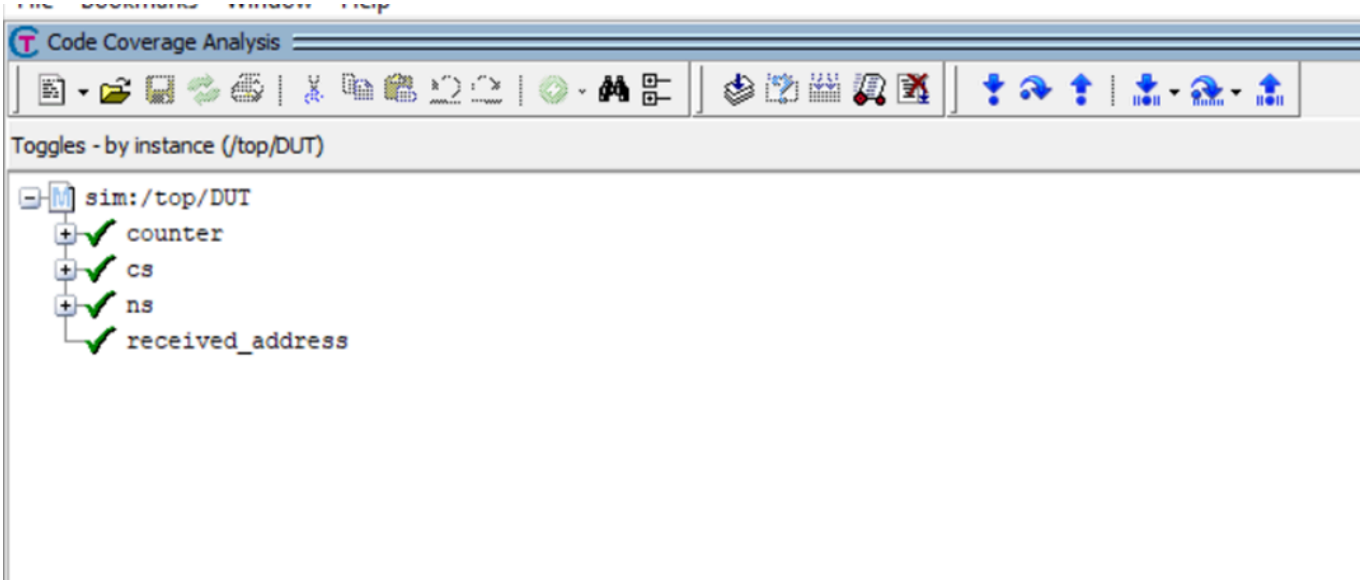



Code Coverage Analysis
Branches - by instance (/top/DUT)
Branch
SPI_slave.sv
20 if (~ss_if.rst_n) begin
23 else begin
29 case (cs)
30 IDLE : begin
31 if (ss_if.SS_n)
33 else
36 CHK_CMD : begin
37 if (ss_if.SS_n)
39 else begin
40 if (~ss_if.MOSI)
42 else begin
46 if (received_address)
48 else
53 WRITE : begin
54 if (ss_if.SS_n)
56 else
59 READ_ADD : begin
60 if (ss_if.SS_n)
62 else
65 READ_DATA : begin
66 if (ss_if.SS_n)
68 else
75 if (~ss_if.rst_n) begin
82 else begin
83 case (cs)
84 IDLE : begin
87 CHK_CMD : begin
90 WRITE : begin
91 if (counter > 0) begin
95 else begin
99 READ_ADD : begin
100 if (counter > 0) begin
104 else begin
109 READ_DATA : begin
110 if (ss_if.tx_valid) begin
114 if (counter > 0) begin


Code Coverage Analysis
Statements - by instance (/top/DUT)
Statement
SPI_slave.sv
19 always @(posedge ss_if.clk) begin
21 cs <= IDLE;
24 cs <= ns;
28 always @(*) begin
32 ns = IDLE;
34 ns = CHK_CMD;
38 ns = IDLE;
41 ns = WRITE;
47 ns = READ_DATA; // Fixed: should go to READ_DATA when address received
49 ns = READ_ADD; // Fixed: should go to READ_ADD when no address received
55 ns = IDLE;
57 ns = WRITE;
61 ns = IDLE;
63 ns = READ_ADD;
67 ns = IDLE;
69 ns = READ_DATA;
74 always @(posedge ss_if.clk) begin
76 ss_if.rx_data <= 0;
77 ss_if.rx_valid <= 0;
78 received_address <= 0;
79 ss_if.MISO <= 0;
80 counter <= 0; // ORIGINAL BUG: Counter not reset - FIXED: Added counter reset
85 ss_if.rx_valid <= 0;
88 counter <= 10;
92 ss_if.rx_data[counter-1] <= ss_if.MOSI;
93 counter <= counter - 1;
96 ss_if.rx_valid <= 1;
101 ss_if.rx_data[counter-1] <= ss_if.MOSI;
102 counter <= counter - 1;
105 ss_if.rx_valid <= 1;
106 received_address <= 1;
115 ss_if.MISO <= ss_if.tx_data[counter-1];
116 counter <= counter - 1;
119 received_address <= 0;
120 ss_if.rx_valid <= 0; // Fixed: rx_valid reset at appropriate time
125 ss_if.rx_data[counter-1] <= ss_if.MOSI;


FSMs - by instance (/top/DUT)
cs
IDLE (0)
CHK_CMD (2)
READ_ADD (3)
READ_DATA (4)
WRITE (1)





Full Coverage report is attached in zip file